Autonomy II Final Project Report - EKF SLAM

J. Poirier

May 7, 2026

Abstract

This report details the development and implementation of a ROS2-based extended Kalman filter simultaneous localization and mapping (EKF SLAM) simulation. RViz2 visualization is used for simulation of a tri-wheel omnidirectional robot. The successful integration of sensor simulation files, URDF descriptions, ODE-based controllers, filters, and ROS2 structures demonstrates the practical application of Kalman filters, self-localization, estimation, and mapping in robotic systems. Additionally, explorations into more advanced techniques and libraries, like ORB-SLAM3, gave insight into modern methods, use cases, and aspects to be aware of that may cause inconsistencies or instabilities in real-world applications.

Contents

1 EKF SLAM **3**
- 1.1 ROS2 Package Setup . . . 3
- 1.2 Nodes . . . 4
  - 1.2.1 fake_lidar_scan_world.py . . . 4
  - 1.2.2 ode_tf_broadcaster.py . . . 4
  - 1.2.3 omni_ode_plant.py . . . 4
  - 1.2.4 world_markers.py . . . 4
  - 1.2.5 slam_classes.py . . . 4
  - 1.2.6 ekf_node.py . . . 5
  - 1.2.7 Topic Summary Table . . . 6
- 1.3 Running the Package . . . 7
- 1.4 Results . . . 7
- 1.5 Difficulties and Resolutions . . . 10

2 ORB-SLAM3 Exploration **10**
- 2.1 Procedure . . . 10
- 2.2 Results . . . 11
- 2.3 Difficulties and Resolutions . . . 13

3 Conclusion **13**

4 Appendix **14**
- 4.1 Code . . . 14
  - 4.1.1 simple_world.yaml . . . 14
  - 4.1.2 ekf_node.py . . . 14
  - 4.1.3 fake_lidar_scan_world.py . . . 17
  - 4.1.4 ode_tf_broadcaster.py . . . 21
  - 4.1.5 omni_ode_plant.py . . . 23
  - 4.1.6 slam_classes.py . . . 25
  - 4.1.7 world_markers.py . . . 31
  - 4.1.8 launch.py . . . 33
  - 4.1.9 slam_view.rviz . . . 35
  - 4.1.10 tri_wheel.urdf . . . 39
  - 4.1.11 package.xml . . . 44
  - 4.1.12 setup.py . . . 44

1 EKF SLAM

This portion of the project consists of creating an EKF SLAM simulation with occupancy grid mapping using ROS2 and Rviz. The simulation uses a tri-wheel omni-directional robot. Much of the framework for this robot simulation was developed previously in the course, and will not be detailed in depth in this report. Additionally, several files were provided by the instructor and needed minimal adjustment.

1.1 ROS2 Package Setup

The first step is to create a new ROS2 package to implement everything in. The following steps can be followed to achieve this:

1. Open your WSL/Ubuntu Terminal
2. Navigate to the ROS2 workspace source folder. In this case, this command would be used: `cd ~/ros2_course_ws/src`
3. Run the following command to create a new package with the required dependencies: `ros2 pkg create --build-type ament_python tri_wheel_slam --dependencies rclpy std_m`

With that, the package is set up and can be worked with. Additionally, the package should end with the below file structure, excluding the folders and files automatically created by ROS. All files that need to be manually created can be found in the appendix.

```
ekf_slam/
├── config/
│   └── simple_world.yaml
├── ekf_slam/
│   ├── ekf_node.py
│   ├── fake_lidar_scan_world.py
│   ├── ode_tf_broadcaster.py
│   ├── omni_ode_plant.py
│   ├── slam_classes.py
│   └── world_markers.py
├── launch/
│   └── launch.py
├── rviz/
│   └── slam_view.rviz
├── urdf/
│   └── tri_wheel.urdf
├── package.xml
└── setup.py
```

The launch, rviz, urdf, package, data, and setup files are not documented in this report aside from being provided in the appendix, as they are standard practice for ROS2 and have no special additions. The node files are documented in **Section** 1.2.

1.2 Nodes

With the EKF SLAM package properly set up, nodes can be developed and implemented. The majority of the nodes, with the exception of ekf_node.py and a portion of slam_classes.py, were provided directly by the instructor either from previous assignments or for this project specifically. These files are provided in the appendix and will not be explained in depth.

1.2.1 fake_lidar_scan_world.py

This node was provided by the instructor for this project. The node generates artificial lidar scans by ray-casting against world geometry defined by the simple_world.yaml file. The node gets the odometry to lidar link transform from the `/tf` topic to get the sensor's current world-frame position and heading, then casts rays which "collide" with wall segments. Results are published to the `/scan` topic.

1.2.2 ode_tf_broadcaster.py

This node was provided by the instructor from a previous assignment of the course. This node bridges the plant pose output into the ROS2 TF tree. The node subscribes to `/pose` and broadcasts a transform on the `tf` topic for the odometry to base footprint transformation, allowing the robot to know where it is in the world frame.

1.2.3 omni_ode_plant.py

This node was provided by the instructor from a previous assignment of the course. This node simulates the tri-wheel robot kinematics. The node subscribes to the `cmd_vel` topic and uses the omnidirectional motion equations with Euler integration. The resulting ground-truth pose is published to the `/pose` topic.

1.2.4 world_markers.py

This node was provided by the instructor for this project. The node loads the wall and obstacle geometry from `simple_world.yaml` and publishes marker data to the `world_markers` topic, allowing RViz to display the ground-truth obstacle and wall locations for reference against the occupancy map.

1.2.5 slam_classes.py

This script was adapted from a script provided by the instructor for this project (originally named `EE5560_EKF_SLAM_Occupancy_ICP.py`). This script, although not a node itself, contains the vast majority of the mathematical and algorithmic structures for SLAM. This script provides the EKFPose, OccupancyGridMap, and icp_scan_match_pose functions, as well as the world constants to `ekf_node`. The script was originally written as a standalone Pygame simulation, and was adapted to act as a function library for the package.

1.2.6 ekf_node.py

This node was created for this project, and is the core SLAM node. This node maintains the pose estimate via EKF and an occupancy map of the environment. This node runs the EKF intitialization, initializing on the origin (set to center) of the occupancy map. Then, the node manages the prediction step, the correction step, and the mapping of the occupancy map. The node subscribes to the `/pose` topic, and on every message, the node computes the odometry delta in the world frame and transforms to the robot frame. The node also subscribes to the `/scan` topic and on every message converts raw ranges to beam objects. Once the occupancy map reaches a certain number of occupied cells, ICP (iterative closest point) scan matching begins. The current scan's hit points are transformed into the world frame using the predicted pose, then aligned to the set of occupied map cells using ICP with SVD-based rigid body solving. If the result does not have a significant jump in the transformation (the result is plausible), it is accepted as a pose measurement. That pose measurement is used for the EKF update. Finally, mapping is handled. Once the pose is corrected, the beam objects are used to update the occupancy map using log-odds ray tracing. Each beam marks all cells between the robot and the scan hit as unoccupied, and the cell where the hit was as occupied. The map is then published to RViz for visualization.

Process Model

The process model is a nonlinear omnidirectional kinematic model. On each `/pose` callback, the world-frame odometry delta is rotated into the robot's local body frame and propagated as:

$$x_{k+1} = x_k + \cos\theta_k \cdot \delta x_{\text{local}} - \sin\theta_k \cdot \delta y_{\text{local}} \tag{1}$$

$$y_{k+1} = y_k + \sin\theta_k \cdot \delta x_{\text{local}} + \cos\theta_k \cdot \delta y_{\text{local}} \tag{2}$$

$$\theta_{k+1} = \theta_k + \delta\theta \tag{3}$$

The Jacobian G linearizes this model around the current heading:

$$G = \begin{bmatrix} 1 & 0 & -\sin\theta_k \cdot \delta x_{\text{local}} - \cos\theta_k \cdot \delta y_{\text{local}} \\ 0 & 1 & \cos\theta_k \cdot \delta x_{\text{local}} - \sin\theta_k \cdot \delta y_{\text{local}} \\ 0 & 0 & 1 \end{bmatrix} \tag{4}$$

The input noise is mapped into the state frame through the rotation matrix V:

$$V = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{5}$$

and the covariance is propagated as:

$$\Sigma_{k+1} = G\,\Sigma_k\,G^\top + V\,Q\,V^\top \tag{6}$$

Process Noise Covariance Q

A fixed diagonal matrix reflecting an assumed odometry uncertainty of $\pm 5\,\text{cm}$ in translation and $\pm 2^\circ$ in heading per step:

$$Q = \begin{bmatrix} 0.05^2 & 0 & 0 \\ 0 & 0.05^2 & 0 \\ 0 & 0 & (2^\circ)^2 \end{bmatrix} = \begin{bmatrix} 0.0025 & 0 & 0 \\ 0 & 0.0025 & 0 \\ 0 & 0 & 0.00122 \end{bmatrix} \tag{7}$$

Measurement Model

The measurement model uses a full-pose observation with $H = I_3$, meaning the ICP scan matcher returns a direct estimate of the full state $[x,\, y,\, \theta]^\top$. The model is:

$$\nu = z_{\text{pose}} - \mu, \qquad \nu_\theta = \text{wrap}(\nu_\theta) \tag{8}$$

The Kalman gain and Joseph-form covariance updates are:

$$K = \Sigma \left(\Sigma + R\right)^{-1} \tag{9}$$

$$\mu \leftarrow \mu + K\nu \tag{10}$$

$$\Sigma \leftarrow (I - KH)\,\Sigma\,(I - KH)^\top + K\,R\,K^\top \tag{11}$$

The correction is applied only when ICP meets certain criteria in the number of correspondents, the position correction limit, and the heading correction limit.

Measurement Noise Covariance R

A fixed diagonal matrix reflecting the expected ICP scan-match accuracy of $\pm 4.5\,\text{cm}$ in position and $\pm 1.5^\circ$ in heading:

$$R = \begin{bmatrix} 0.045^2 & 0 & 0 \\ 0 & 0.045^2 & 0 \\ 0 & 0 & (1.5^\circ)^2 \end{bmatrix} = \begin{bmatrix} 0.002025 & 0 & 0 \\ 0 & 0.002025 & 0 \\ 0 & 0 & 0.000685 \end{bmatrix} \tag{12}$$

Since $R < Q$ in all diagonal entries, the filter prioritizes ICP corrections over dead-reckoning whenever a valid match is found.

Initial Covariance Σ_0

The robot is initialised at the origin with small, but nonzero uncertainty, matching the scale of the process noise:

$$\Sigma_0 = \begin{bmatrix} 0.05^2 & 0 & 0 \\ 0 & 0.05^2 & 0 \\ 0 & 0 & (2^\circ)^2 \end{bmatrix} \tag{13}$$

1.2.7 Topic Summary Table

The nodes described above create three distinct pipelines. The first is the simulation pipeline, which uses the plant, TF broadcaster, and the URDF file to simulate the robot and its

movements. The second is the sensing pipeline, which runs the artificial lidar and mapping procedures. The third is the control pipeline, which acts as the controller for the simulation, allowing the robot to be controlled via the 2D Goal Pose tool in RViz. **Table** 1 below gives a table summary of the topics.

Table 1: ROS2 Topic Summary

Topic	**Message Type**	**Publisher → Subscriber(s)**	**Role**
`/cmd_vel`	`geometry_msgs/Twist`	`omni_go_to_goal` → `omni_ode_plant`	Velocity commands
`/pose`	`turtlesim/Pose`	`omni_ode_plant` → `ode_tf_broadcaster`, `ekf_node`, `omni_go_to_goal`	Ground-truth robot pose
`/tf`	TF2 transforms	`ode_tf_broadcaster`, `robot_state_publisher` → `fake_lidar_scan_world`	Full TF tree (odom→lidar_link)
`/joint_states`	`sensor_msgs/JointState`	`joint_state_publisher` → `robot_state_publisher`	URDF joint values
`/scan`	`sensor_msgs/LaserScan`	`fake_lidar_scan_world` → `ekf_node`	Synthetic lidar ranges
`/map`	`nav_msgs/OccupancyGrid`	`ekf_node` → RViz2	Live occupancy map
`/world_markers`	`visualization_msgs/Marker`	`world_markers` → RViz2	Ground-truth wall overlay
`/goal_pose`	`geometry_msgs/PoseStamped`	RViz2 → `omni_go_to_goal`	User-clicked navigation goal

1.3 Running the Package

To run the package, first run the following commands:

1. `cd ~/ros_course_ws`
2. `colcon build`
3. `source install/setup.bash`
4. `ros2 launch ekf_slam launch.py`

This will open the RViz window. While in this window, you can use the 2D Goal Pose tool to command the robot to move around the area. As the robot moves, the occupancy map will populate and update. These functions are shown in **Section** 1.4.

1.4 Results

The results for this problem were very successful. Using the frameworks that were built up over the course of this class and its assignments, a full simulated implementation of SLAM in ROS2 was feasible and successful. The RViz window that opens from running the package

is shown in **Figure** 1, along with a few evolutions of the view after moving the robot in **Figure** 2.

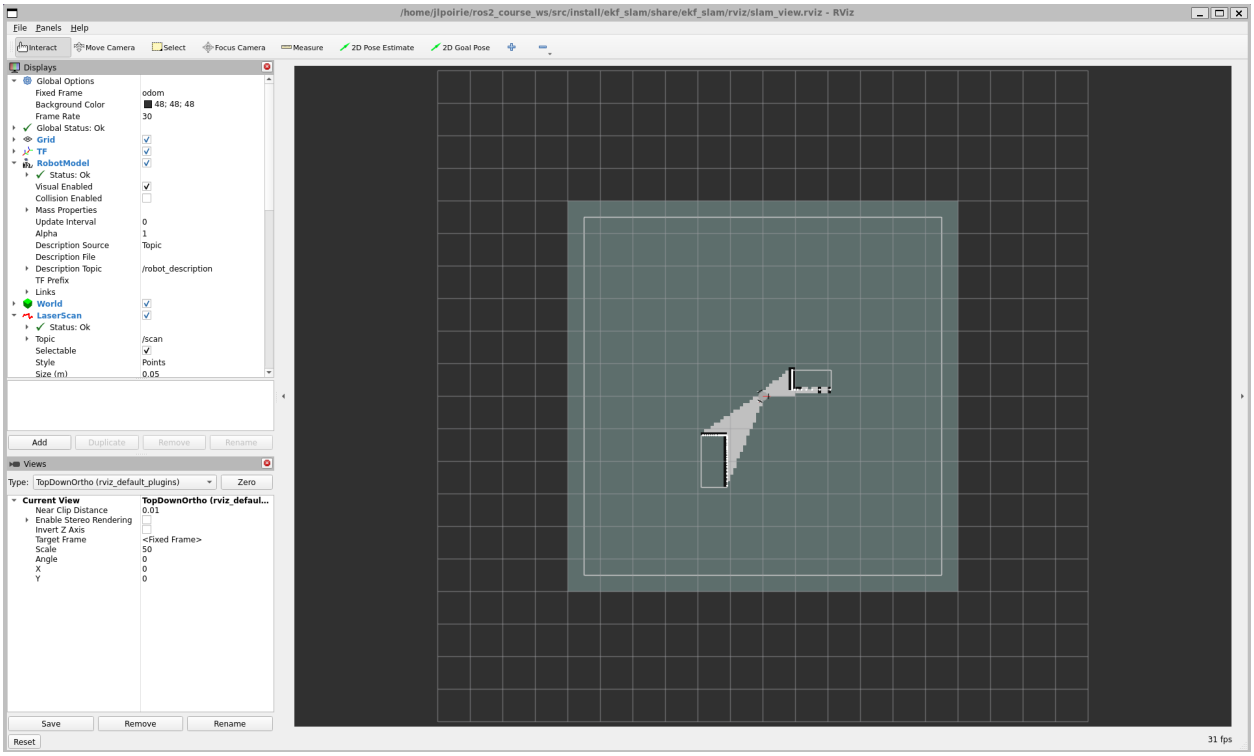


Figure 1: RViz2 EKF SLAM Simulation with Occupancy Map

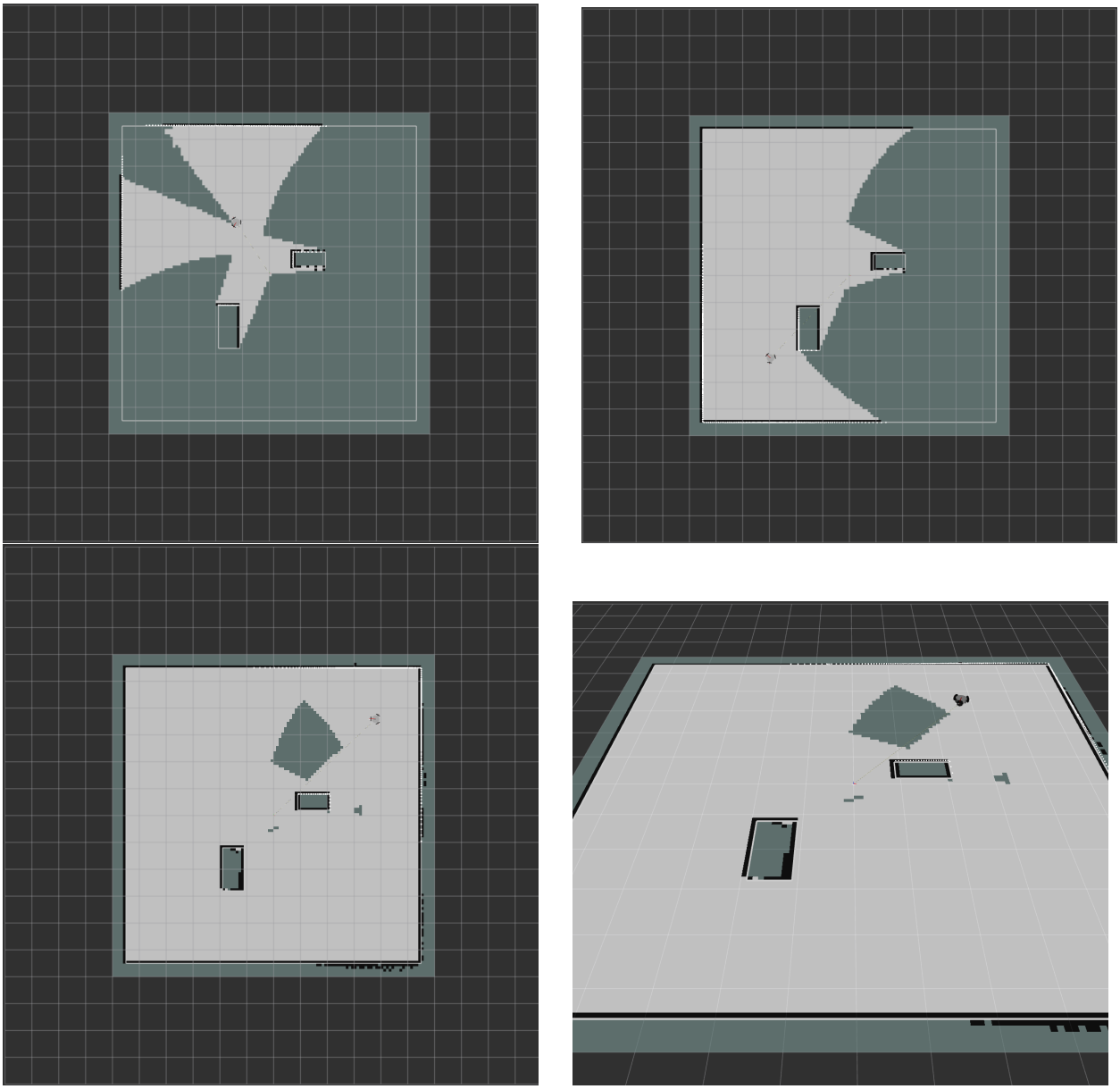


Figure 2: RViz2 EKF SLAM Simulation with Occupancy Map Evolution

The occupancy map properly initializes and updates as the robot moves, using the calculated confidences to classify cells within the map. Additionally, **Figure** 3 shows the RQT Graph of the full system.

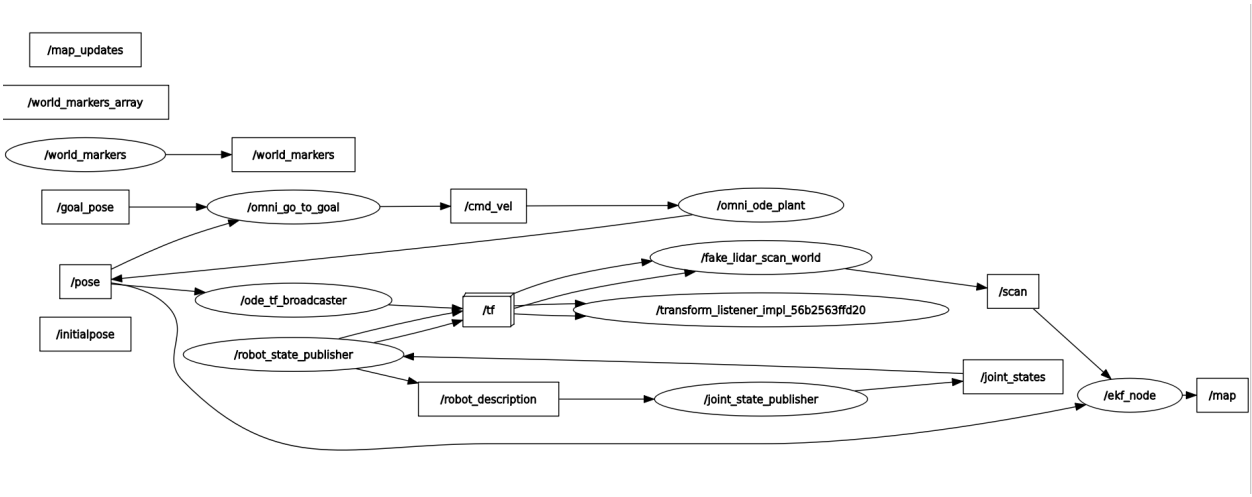


Figure 3: RViz2 EKF SLAM Simulation RQT Graph

The RQT Graph coincides with **Table** 1, showing the flow of information throughout all the nodes.

1.5 Difficulties and Resolutions

Overall, development of this project was smooth with minimal errors. Adapting old nodes and provided nodes took only minor changes, and creating `ekf_node` was the primary challenge. Mapping errors and localization drift are present, as can be seen in **Figure** 2. When the robot collects inconsistent data, but it is not so much of an outlier that it gets rejected, the estimation becomes "blurry" and less accurate. There were also sensor range constraints that limited the amount of data the robot could collect at any given position, allowing for higher position prediction error. If improvements were to be made to this simulation, a larger lidar range, higher lidar frequency, and higher occupancy grid resolution could be used. Of course, if implemented in real-world use, hardware limitations would be a significant concern, and even in simulation, computational power is always a factor.

2 ORB-SLAM3 Exploration

This portion of the project consists of exploring the pre-existing ORB-SLAM3 package for ROS2. The purpose of this is to gain experience with modern, more advanced localization and estimation techniques.

2.1 Procedure

The procedure described in the provided project document was followed for this problem, and no major issues arose.

2.2 Results

The results of this exercise showed that modern SLAM techniques can be extremely powerful but can still suffer from non-ideal conditions. **Figures** 4 **and** 5 below show the results of ORB-SLAM3 using the provided video.

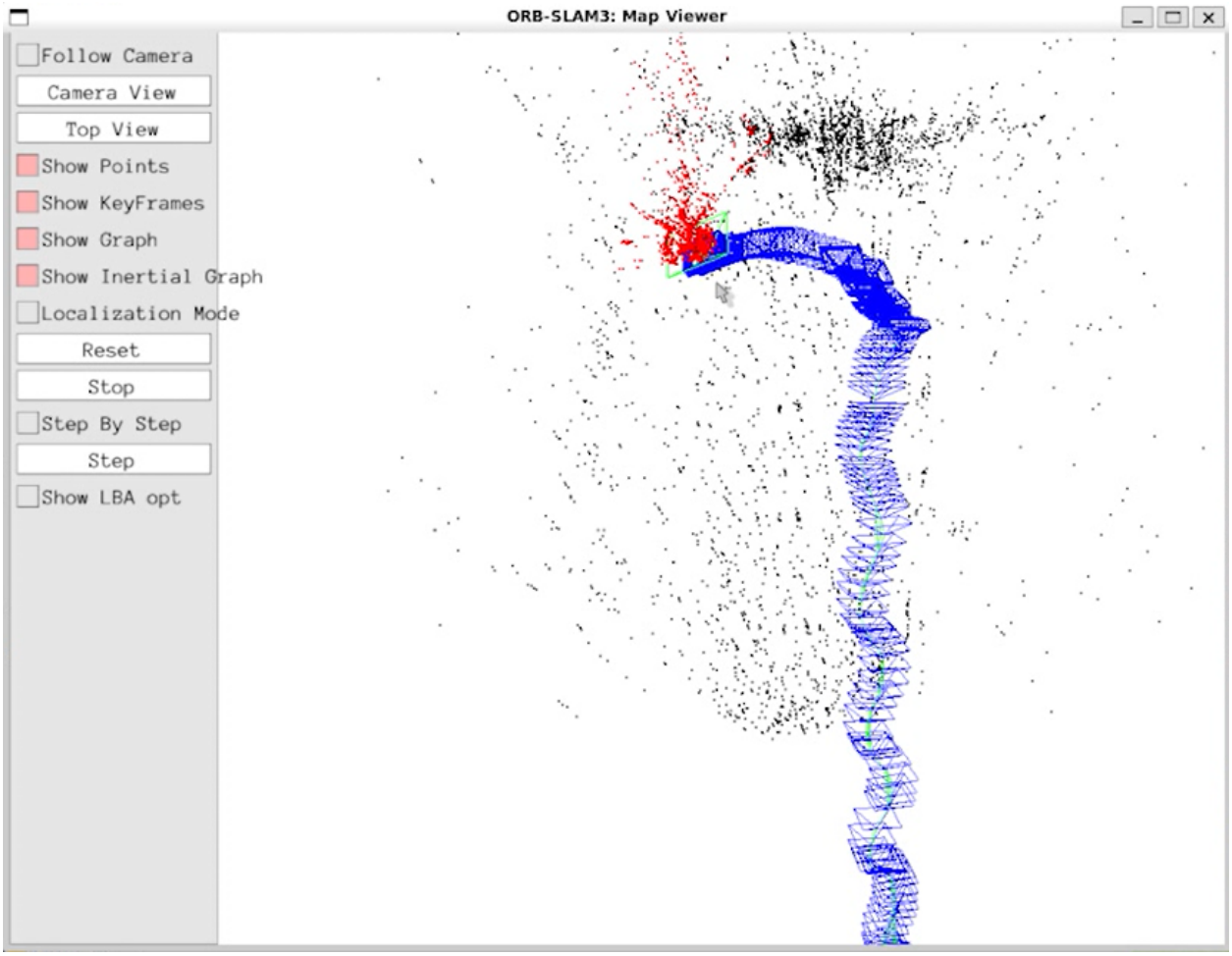


Figure 4: ORB-SLAM3 Package Self-Localization Map

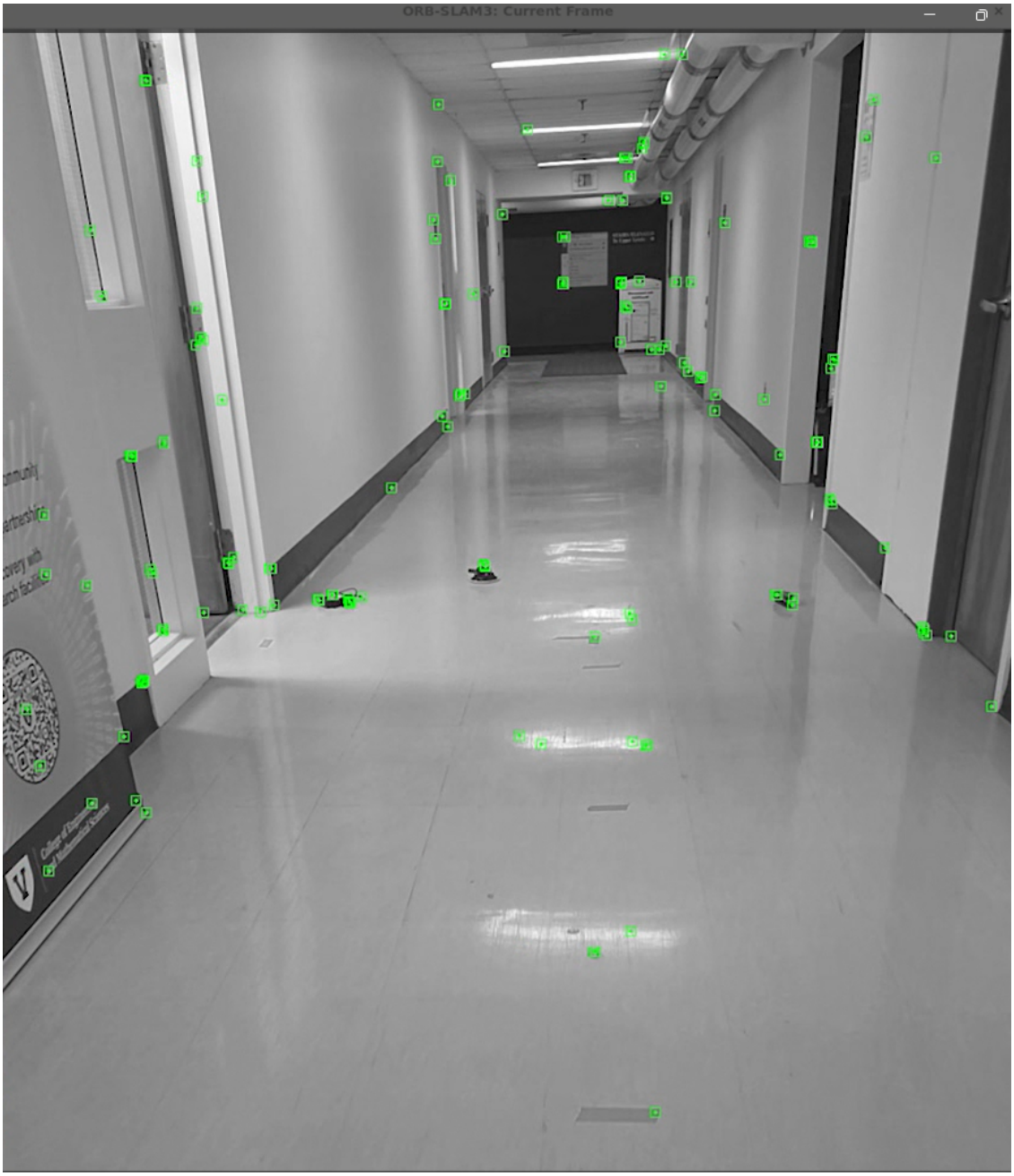


Figure 5: ORB-SLAM3 Data Video with Point Tracking Indicators

From **Figures** 4 **and** 5, it is clear that ORB-SLAM3 has a high level of sophistication, but still suffers from drift, suboptimal visual conditions, and other factors that significantly degrade performance. ORB-SLAM3 initialized successfully, shortly after running the start command for the package. Notably, running the package on a powerful desktop PC versus a laptop yielded significant initialization time differences as well as actual performance differences. ORB-SLAM3 uses much more computing power than the EKF SLAM that was implemented in problem 1, and thus benefits more from improved hardware.

Tracking was not stable throughout the video, having several issues occur throughout the

course of the video. Most notably, the tracking would often be completely lost on turns and corners, occasionally resulting in the camera frame effectively thinking it has been reset in the world frame. When the camera turns, it is usually up against a wall, meaning the camera field of view is significantly hindered, giving far fewer trackable points. In the opposite case, where any trackable points are far away, the changes in camera position will be estimated to be much smaller, and thus the estimated position may not complete a full 90 degree turn or may lose tracking entirely. Additionally, the turns happen quickly, and since this process takes significant computing power, moving too fast combined with dropped frames in the processing leads to lost tracking. Slower movements, and especially slower turns, would benefit the performance, as well as having a more open space with less walls, but still with more distinct trackable points. The video used in this project attempted to use tape on the ground to give more trackable points. The two issues with this is that the tape was only along the path being walked, so tracking would be lost as you approached the object, and that the tape is effectively two dimensional. Having more markers, and 3 dimensional markers, throughout the environment, not just in the path of travel, would improve performance.

To enable this on a live video feed, the pipeline would only have to be changed slightly. In its current state, ORB-SLAM3 is already applied frame-by-frame, so the only major difference is that it would pull frames from a live video feed instead of a pre-loaded video. However, since the video is not loaded from memory and is instead being live-broadcast, the computational power will suffer.

2.3 Difficulties and Resolutions

One required, but simple error, occurred when trying to compile the package. When building the `orbslam3_video` package, the compiler would throw an error that the `config/` path was missing. This was solved by simply adding an empty folder to the package named `config`. Any other build issues were resolved quickly by clearing any stale builds and cache files and re-compiling the package.

3 Conclusion

Overall, this project was very successful. Files from previously in the course gave great guidance on how to implement the EKF SLAM functionality, and the project document was very well put together and well defined. No major issues came up during development that prevented the completion of this project. The majority of issues came from ROS2 quirks, with strict package formatting structures. The success of the EKF SLAM simulation demonstrates the usefulness of SLAM and similar estimation algorithms, as well as the considerations necessary when implementing such algorithms. Though this project did not have a physical component, the project still gave significant insight into robotics algorithms with computational limitation, and was crucial in the development of ROS skills.

4 Appendix

4.1 Code

4.1.1 simple_world.yaml

```
world:
  walls:
    - [ -5.5, -5.5,  5.5, -5.5 ]
    - [  5.5, -5.5,  5.5,  5.5 ]
    - [  5.5,  5.5, -5.5,  5.5 ]
    - [ -5.5,  5.5, -5.5, -5.5 ]

  boxes:
    - { cx:  1.5,  cy:  0.5,  w: 1.2, h: 0.6 }
    - { cx: -1.5,  cy: -2.0,  w: 0.8, h: 1.6 }
```

4.1.2 ekf_node.py

```
# Node created originally for this project
# John Poirier

#!/usr/bin/env python3
import rclpy
import math
import numpy as np
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy

from turtlesim.msg import Pose
from sensor_msgs.msg import LaserScan
from nav_msgs.msg import OccupancyGrid

from ekf_slam.slam_classes import EKFPose, OccupancyGridMap, Beam,
   WORLD_XMIN, WORLD_YMIN, icp_scan_match_pose, ICP_MIN_MATCHES,
   ICP_R_MEAS_BASE


def wrap_angle(angle):
    return (angle + math.pi) % (2.0 * math.pi) - math.pi

class EKFSlamNode(Node):
    def __init__(self):
        super().__init__('ekf_slam_node')

        # initialize EKF and occupancy grid at origin
        self.ekf = EKFPose(np.array([0.0, 0.0, 0.0], dtype=float))
        self.grid = OccupancyGridMap()
        self.last_pose = None

        # subscribers - ODE plant pose for odometry
        self.pose_sub = self.create_subscription(
            Pose, '/pose', self.pose_cb, 10)
```

```
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_cb, QoSProfile(depth=10,
reliability=ReliabilityPolicy.BEST_EFFORT))

        # publishers
        map_qos = QoSProfile(
            depth=1,
            durability=DurabilityPolicy.VOLATILE,
            reliability=ReliabilityPolicy.RELIABLE
        )
        self.map_pub = self.create_publisher(OccupancyGrid, '/map',
map_qos)

        # publish map every second
        self.create_timer(1.0, self.publish_map)

        self.get_logger().info("EKF SLAM Node started. Waiting for /pose
and /scan")

    def pose_cb(self, msg: Pose):
        """
        EKF Prediction Step: Calculate the delta between the current and
last
        odometry pose, convert it to the robot's local frame, and predict.
        """
        if self.last_pose is None:
            self.last_pose = msg
            return

        # calculate changes in global frame
        dx_world = msg.x - self.last_pose.x
        dy_world = msg.y - self.last_pose.y
        dtheta = wrap_angle(msg.theta - self.last_pose.theta)

        # convert to robot local frame
        c = math.cos(self.last_pose.theta)
        s = math.sin(self.last_pose.theta)
        dx_local = c * dx_world + s * dy_world
        dy_local = -s * dx_world + c * dy_world

        odom = np.array([dx_local, dy_local, dtheta], dtype=float)

        # base odometry noise covariance
        odom_cov = np.diag([0.05**2, 0.05**2, math.radians(2.0)**2])

        # EKF prediction
        self.ekf.predict(odom, odom_cov)
        self.last_pose = msg

    def scan_cb(self, msg: LaserScan):
        """
        EKF Correction and Mapping Step.
        1. Convert LaserScan to Beam objects.
```

```
        2. Once the map has enough occupied cells, run ICP to get a
        scan-matched pose estimate.
        3. Feed pose into ekf.correct_pose_measurement().
        4. Update the occupancy grid with the corrected pose.
        """
        # laser scan to beam objects
        beams = []
        for i, r in enumerate(msg.ranges):
            if msg.range_min < r < msg.range_max and not math.isinf(r) and
 not math.isnan(r):
                angle = msg.angle_min + i * msg.angle_increment
                beams.append(Beam(range=r, bearing=angle, hit=True))

        if not beams:
            return

        est_pose = self.ekf.pose()

        # don't correct until above cetain number of occupied cells
        n_occupied = int(np.sum(self.grid.probability() > 0.65))

        if n_occupied >= ICP_MIN_MATCHES * 4:   # wwait until certain
number of occupied cells exist
            matched_pose, score, n_matches = icp_scan_match_pose(
                self.grid, est_pose, beams
            )

            # accept correction under certain conditions
            correction_xy  = float(np.linalg.norm(matched_pose[:2] -
est_pose[:2]))
            correction_th  = abs(wrap_angle(matched_pose[2] - est_pose[2])
)

            if (n_matches >= ICP_MIN_MATCHES
                    and correction_xy  <= 0.45 # reject large jumps
                    and correction_th  <= math.radians(10.0)):

                R_meas = np.diag([0.045**2, 0.045**2, math.radians(1.5)
**2])
                self.ekf.correct_pose_measurement(matched_pose, R_meas)
                est_pose = self.ekf.pose()   # use corrected pose for
mapping

                self.get_logger().debug(
                    f"ICP correction accepted: n={n_matches}, score={score
:.4f}, "
                    f"dxy={correction_xy:.3f}m, dth={math.degrees(
correction_th):.2f}deg"
                )

        # update occ grid with corrected pose
        for beam in beams:
            self.grid.update_from_beam(est_pose, beam)
```

```
    def publish_map(self):
        """
        Translates the internal OccupancyGridMap (log-odds) into a ROS2
   nav_msgs/OccupancyGrid.
        Dimensions and origin are read directly from the grid object to
   always match.
        """
        msg = OccupancyGrid()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'odom'

        prob = self.grid.probability()
        grid_h, grid_w = prob.shape

        res = 0.1 # must match GRID_RES in slam_classes.py
        msg.info.resolution = res
        msg.info.width  = grid_w
        msg.info.height = grid_h

        # center so origin is center cell
        msg.info.origin.position.x = WORLD_XMIN
        msg.info.origin.position.y = WORLD_YMIN
        msg.info.origin.orientation.w = 1.0

        # convert probabilities
        ros_map = np.full(prob.shape, -1, dtype=np.int8)
        ros_map[prob > 0.65] = 100
        ros_map[prob < 0.35] = 0

        # ROS OccupancyGrid is row-major with row 0 at the bottom (y_min)
        msg.data = ros_map.flatten().tolist()
        self.map_pub.publish(msg)

def main():
    rclpy.init()
    node = EKFSlamNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

4.1.3 fake_lidar_scan_world.py

```
#Provided directly from instructor Dr. Duffaut for this project (
   course_ode_plant_urdf_goal_world)

#!/usr/bin/env python3
import math
```

```
import os
import yaml

import rclpy
from rclpy.node import Node

from sensor_msgs.msg import LaserScan
from tf2_ros import Buffer, TransformListener
from tf2_ros import LookupException, ConnectivityException,
   ExtrapolationException


def cross2(ax, ay, bx, by):
    return ax * by - ay * bx


def ray_intersect_segment(px, py, dx, dy, x1, y1, x2, y2):
    """
    Robust 2D intersection:
      Ray:     p + t r,    t >= 0
      Segment q + u s,    u in [0,1]
    where:
      p=(px,py), r=(dx,dy)
      q=(x1,y1), s=(x2-x1,y2-y1)

    t = cross((q-p), s) / cross(r, s)
    u = cross((q-p), r) / cross(r, s)
    """
    sx = x2 - x1
    sy = y2 - y1

    rxs = cross2(dx, dy, sx, sy)
    if abs(rxs) < 1e-12:
        return None  # parallel (or collinear)

    qpx = x1 - px
    qpy = y1 - py

    t = cross2(qpx, qpy, sx, sy) / rxs
    u = cross2(qpx, qpy, dx, dy) / rxs

    if t >= 0.0 and 0.0 <= u <= 1.0:
        return t
    return None



def box_to_segments(cx, cy, w, h):
    x1 = cx - w / 2.0
    x2 = cx + w / 2.0
    y1 = cy - h / 2.0
    y2 = cy + h / 2.0
    return [
        (x1, y1, x2, y1),
```

```
        (x2, y1, x2, y2),
        (x2, y2, x1, y2),
        (x1, y2, x1, y1),
    ]


def yaw_from_quat(qx, qy, qz, qw):
    # standard yaw extraction
    return math.atan2(2.0 * (qw * qz + qx * qy),
                      1.0 - 2.0 * (qy * qy + qz * qz))


class FakeLidarScanWorld(Node):
    """
    Correct frame-consistent fake lidar:
      - Ray casting origin and yaw come from TF(odom -> lidar_link)
      - LaserScan is published in lidar_link frame
      - World geometry is stored in odom/world coordinates
    """

    def __init__(self):
        super().__init__('fake_lidar_scan_world')

        # Parameters
        self.declare_parameter('scan_topic', '/scan')
        self.declare_parameter('lidar_frame', 'lidar_link')
        self.declare_parameter('odom_frame', 'odom')
        self.declare_parameter('world_yaml', '')

        self.declare_parameter('rate_hz', 10.0)
        self.declare_parameter('angle_min', -math.pi)
        self.declare_parameter('angle_max', math.pi)
        self.declare_parameter('num_rays', 360)
        self.declare_parameter('range_min', 0.05)
        self.declare_parameter('range_max', 5.0)

        self.scan_topic = self.get_parameter('scan_topic').value
        self.lidar_frame = self.get_parameter('lidar_frame').value
        self.odom_frame = self.get_parameter('odom_frame').value

        self.rate_hz = float(self.get_parameter('rate_hz').value)
        self.angle_min = float(self.get_parameter('angle_min').value)
        self.angle_max = float(self.get_parameter('angle_max').value)
        self.num_rays = int(self.get_parameter('num_rays').value)
        self.range_min = float(self.get_parameter('range_min').value)
        self.range_max = float(self.get_parameter('range_max').value)

        world_yaml = self.get_parameter('world_yaml').value
        if not world_yaml or not os.path.exists(world_yaml):
            raise RuntimeError(f"world_yaml not found: {world_yaml}")

        self.segments = self.load_world_segments(world_yaml)

        # TF
```

```
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)

        self.pub = self.create_publisher(LaserScan, self.scan_topic, 10)
        self.create_timer(1.0 / self.rate_hz, self.tick)

        self.get_logger().info(
            f"Fake lidar (TF-correct) publishing {self.scan_topic} in
    frame '{self.lidar_frame}' "
            f"using TF {self.odom_frame}->{self.lidar_frame}"
        )

    def load_world_segments(self, yaml_path: str):
        with open(yaml_path, 'r') as f:
            data = yaml.safe_load(f)
        w = data.get('world', {})
        segs = []

        for s in w.get('walls', []):
            if len(s) != 4:
                raise ValueError(f"Invalid wall: {s}")
            segs.append(tuple(float(v) for v in s))

        for b in w.get('boxes', []):
            cx = float(b['cx']); cy = float(b['cy'])
            ww = float(b['w']);  hh = float(b['h'])
            segs.extend(box_to_segments(cx, cy, ww, hh))

        if not segs:
            raise ValueError("World has no walls/boxes.")
        return segs

    def tick(self):
        # lookup TF origin and yaw of lidar in odom
        try:
            tf = self.tf_buffer.lookup_transform(
                self.odom_frame, self.lidar_frame,
                rclpy.time.Time()
            )
        except (LookupException, ConnectivityException,
    ExtrapolationException):
            return

        px = tf.transform.translation.x
        py = tf.transform.translation.y
        q = tf.transform.rotation
        yaw = yaw_from_quat(q.x, q.y, q.z, q.w)

        scan = LaserScan()
        scan.header.stamp = self.get_clock().now().to_msg()
        scan.header.frame_id = self.lidar_frame

        scan.angle_min = self.angle_min
        scan.angle_max = self.angle_max
```

```
        # IMPORTANT: num_rays should span [min,max] inclusive
        if self.num_rays <= 1:
            return
        scan.angle_increment = (self.angle_max - self.angle_min) / float(
    self.num_rays - 1)

        scan.time_increment = 0.0
        scan.scan_time = 1.0 / self.rate_hz
        scan.range_min = self.range_min
        scan.range_max = self.range_max

        ranges = []
        for i in range(self.num_rays):
            ang = self.angle_min + i * scan.angle_increment
            world_ang = yaw + ang

            dx = math.cos(world_ang)
            dy = math.sin(world_ang)

            best = None
            for (x1, y1, x2, y2) in self.segments:
                t = ray_intersect_segment(px, py, dx, dy, x1, y1, x2, y2)
                if t is not None and (best is None or t < best):
                    best = t

            if best is None or best > self.range_max:
                ranges.append(float('inf'))
            else:
                ranges.append(max(best, self.range_min))

        scan.ranges = ranges
        self.pub.publish(scan)


def main():
    rclpy.init()
    node = FakeLidarScanWorld()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

4.1.4 ode_tf_broadcaster.py

```
#Provided directly from instructor Dr. Duffaut from HW5 solutions

#!/usr/bin/env python3
import math
import rclpy
from rclpy.node import Node
```

```
from turtlesim.msg import Pose
from geometry_msgs.msg import TransformStamped
from tf2_ros import TransformBroadcaster

def quat_from_yaw(yaw: float):
    half = 0.5 * yaw
    return (0.0, 0.0, math.sin(half), math.cos(half))

class ODETFBroadcaster(Node):
    def __init__(self):
        super().__init__('ode_tf_broadcaster')
        self.declare_parameter('pose_topic', '/pose')
        self.declare_parameter('odom_frame', 'odom')
        self.declare_parameter('base_frame', 'base_footprint')

        pose_topic = self.get_parameter('pose_topic').value
        self.odom_frame = self.get_parameter('odom_frame').value
        self.base_frame = self.get_parameter('base_frame').value

        self.tf_broadcaster = TransformBroadcaster(self)
        self.create_subscription(Pose, pose_topic, self.pose_cb, 10)

        self.get_logger().info(f"Broadcasting TF {self.odom_frame}->{self.
   base_frame} from {pose_topic}")

    def pose_cb(self, msg: Pose):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = self.odom_frame
        t.child_frame_id = self.base_frame

        t.transform.translation.x = float(msg.x)
        t.transform.translation.y = float(msg.y)
        t.transform.translation.z = 0.0

        qx, qy, qz, qw = quat_from_yaw(float(msg.theta))
        t.transform.rotation.x = qx
        t.transform.rotation.y = qy
        t.transform.rotation.z = qz
        t.transform.rotation.w = qw

        self.tf_broadcaster.sendTransform(t)

def main():
    rclpy.init()
    node = ODETFBroadcaster()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

4.1.5 omni_ode_plant.py

```
#Provided directly from instructor Dr. Duffaut in HW5 solutions


#!/usr/bin/env python3
import math
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose

def wrap_to_pi(a: float) -> float:
    while a > math.pi:
        a -= 2.0 * math.pi
    while a <= -math.pi:
        a += 2.0 * math.pi
    return a

class OmniODEPlant(Node):
    """
    ODE plant for a tri-wheel robot.
    Subscribes:
      /cmd_vel (geometry_msgs/Twist)  [vx, vy in base frame; omega about z
   ]
    Publishes:
      /pose (turtlesim/Pose)          [x, y, theta] in world (odom) frame
    Kinematics:
      xdot = vx*cos(theta) - vy*sin(theta)
      ydot = vx*sin(theta) + vy*cos(theta)
      thetadot = omega
    """
    def __init__(self):
        super().__init__('omni_ode_plant')
        self.declare_parameter('dt', 0.02)
        self.declare_parameter('x0', 0.0)
        self.declare_parameter('y0', 0.0)
        self.declare_parameter('theta0', 0.0)
        self.declare_parameter('vx_sat', 1.0)
        self.declare_parameter('vy_sat', 1.0)
        self.declare_parameter('w_sat', 3.0)

        self.declare_parameter('cmd_topic', '/cmd_vel')
        self.declare_parameter('pose_topic', '/pose')

        self.dt = float(self.get_parameter('dt').value)
        self.x = float(self.get_parameter('x0').value)
        self.y = float(self.get_parameter('y0').value)
        self.theta = float(self.get_parameter('theta0').value)

        self.vx = 0.0
        self.vy = 0.0
        self.w = 0.0
```

```
        self.vx_sat = float(self.get_parameter('vx_sat').value)
        self.vy_sat = float(self.get_parameter('vy_sat').value)
        self.w_sat = float(self.get_parameter('w_sat').value)

        cmd_topic = self.get_parameter('cmd_topic').value
        pose_topic = self.get_parameter('pose_topic').value

        self.create_subscription(Twist, cmd_topic, self.cmd_cb, 10)
        self.pub_pose = self.create_publisher(Pose, pose_topic, 10)
        self.create_timer(self.dt, self.step)

        self.get_logger().info(f"TriWheelODEPlant started. cmd: {cmd_topic
    } -> pose: {pose_topic}")

    def cmd_cb(self, msg: Twist):
        vx = float(msg.linear.x)
        vy = float(msg.linear.y)
        w = float(msg.angular.z)

        self.vx = max(min(vx, self.vx_sat), -self.vx_sat)
        self.vy = max(min(vy, self.vy_sat), -self.vy_sat)
        self.w = max(min(w, self.w_sat), -self.w_sat)

    def step(self):
        c = math.cos(self.theta)
        s = math.sin(self.theta)

        # body -> world
        xdot = self.vx * c - self.vy * s
        ydot = self.vx * s + self.vy * c

        self.x += self.dt * xdot
        self.y += self.dt * ydot
        self.theta = wrap_to_pi(self.theta + self.dt * self.w)

        pose = Pose()
        pose.x = float(self.x)
        pose.y = float(self.y)
        pose.theta = float(self.theta)

        # These fields are "turtlesim-ish"; we repurpose them
        pose.linear_velocity = float(math.hypot(self.vx, self.vy))
        pose.angular_velocity = float(self.w)

        self.pub_pose.publish(pose)

def main():
    rclpy.init()
    node = OmniODEPlant()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
```

```
    main()
```

4.1.6 slam_classes.py

```
# File provided directly from instructor Dr. Duffaut for this project (
   EE5560_EKF_SLAM_Occupancy_ICP.py)
# Modified and stripped to only what ekf_node.py needs

from __future__ import annotations

import math
from dataclasses import dataclass

import numpy as np


# world and grid constants
WORLD_XMIN = -6.0
WORLD_XMAX =  6.0
WORLD_YMIN = -6.0
WORLD_YMAX =  6.0

GRID_RES = 0.1
GRID_W = int(math.ceil((WORLD_XMAX - WORLD_XMIN) / GRID_RES))
GRID_H = int(math.ceil((WORLD_YMAX - WORLD_YMIN) / GRID_RES))

# occupancy grid update values
LOG_ODDS_OCC = 0.55
LOG_ODDS_FREE = -0.22
LOG_ODDS_MIN = -4.0
LOG_ODDS_MAX =  4.0

MAX_RANGE = 6.5

# ICP scan matching parameters
ICP_MAX_ITER            = 8
ICP_MIN_MATCHES         = 7
ICP_MAX_CORRESP_DIST    = 0.55 #meters
ICP_MAX_TRANSLATION_STEP = 0.18 #meters per ICP iteration
ICP_MAX_ROTATION_STEP   = math.radians(4.0)
ICP_OCC_LOGODDS_THRESHOLD = 0.75
ICP_TARGET_DOWNSAMPLE   = 2
ICP_R_MEAS_BASE = np.diag([0.045 ** 2, 0.045 ** 2, math.radians(1.5) **
   2])

#====================================
# functions
#====================================

def wrap_angle(angle: float) -> float:
    return (angle + math.pi) % (2.0 * math.pi) - math.pi


def world_to_grid(x: float, y: float) -> tuple[int, int]:
```

```
    gx = int(round((x - WORLD_XMIN) / GRID_RES - 0.5))
    gy = int(round((y - WORLD_YMIN) / GRID_RES - 0.5))
    return gx, gy


def inside_grid(gx: int, gy: int) -> bool:
    return 0 <= gx < GRID_W and 0 <= gy < GRID_H


def bresenham_cells(gx0: int, gy0: int, gx1: int, gy1: int):
    """Enumerate all grid cells along the line from (gx0,gy0) to (gx1,gy1)
    """
    x0, y0 = gx0, gy0
    x1, y1 = gx1, gy1
    dx = abs(x1 - x0)
    sx = 1 if x0 < x1 else -1
    dy = -abs(y1 - y0)
    sy = 1 if y0 < y1 else -1
    err = dx + dy
    cells = []
    while True:
        cells.append((x0, y0))
        if x0 == x1 and y0 == y1:
            break
        e2 = 2 * err
        if e2 >= dy:
            err += dy
            x0 += sx
        if e2 <= dx:
            err += dx
            y0 += sy
    return cells


#==================================
# data containers
#==================================

@dataclass
class Beam:
    range:   float
    bearing: float
    hit:     bool


#==================================
# EKF pose estimator
#==================================

class EKFPose:
    def __init__(self, pose: np.ndarray):
        self.mu    = pose.astype(float).copy()
        self.Sigma = np.diag([0.05 ** 2, 0.05 ** 2, math.radians(2.0) **
    2])
```

```
    def pose(self) -> np.ndarray:
        return self.mu.copy()

    def predict(self, odom: np.ndarray, odom_noise: np.ndarray) -> None:
        """
        EKF prediction step using a local-frame odometry input [dx_local,
dy_local, dtheta].
        Propagates the state mean through the nonlinear motion model and
linearises
        the Jacobian G around the current heading to update the covariance
.
        """
        dx_local, dy_local, dtheta = odom
        x, y, theta = self.mu
        c = math.cos(theta)
        s = math.sin(theta)

        # propogate mean
        self.mu[0] = x + c * dx_local - s * dy_local
        self.mu[1] = y + s * dx_local + c * dy_local
        self.mu[2] = wrap_angle(theta + dtheta)

        # Motion Jacobian w.r.t. state (linearised around current theta)
        G = np.eye(3)
        G[0, 2] = -s * dx_local - c * dy_local
        G[1, 2] =  c * dx_local - s * dy_local

        # Input noise mapped through rotation matrix V
        V = np.array([[c, -s, 0.0], [s, c, 0.0], [0.0, 0.0, 1.0]], dtype=
float)
        R = V @ odom_noise @ V.T
        self.Sigma = G @ self.Sigma @ G.T + R
        self.Sigma = 0.5 * (self.Sigma + self.Sigma.T)   # enforce
symmetry

    def correct_pose_measurement(self, z_pose: np.ndarray, R_meas: np.
ndarray) -> None:
        """
        EKF correction step with a full-pose measurement (H = I).
        Uses the Joseph form for numerical stability of the covariance
update.
        """
        H = np.eye(3)
        innovation    = z_pose - self.mu
        innovation[2] = wrap_angle(innovation[2])
        S = self.Sigma + R_meas
        try:
            K = self.Sigma @ np.linalg.inv(S)
        except np.linalg.LinAlgError:
            return
        self.mu    = self.mu + K @ innovation
        self.mu[2] = wrap_angle(self.mu[2])
        I          = np.eye(3)
```

```
        self.Sigma = (I - K @ H) @ self.Sigma @ (I - K @ H).T + K @ R_meas
    @ K.T
        self.Sigma = 0.5 * (self.Sigma + self.Sigma.T)


# ========================================
# Occupancy grid map
# ========================================

class OccupancyGridMap:
    def __init__(self):
        self.log_odds = np.zeros((GRID_H, GRID_W), dtype=float)

    def probability(self) -> np.ndarray:
        """Convert log-odds array to probabilities in [0, 1]."""
        return 1.0 / (1.0 + np.exp(-self.log_odds))

    def update_from_beam(self, pose: np.ndarray, beam: Beam) -> None:
        """
        Update log-odds along a single lidar beam using Bresenham ray
   tracing.
        All cells between the robot and the hit point are marked free;
        the terminal cell (if a hit) is marked occupied.
        """
        x, y, theta = pose
        beam_angle = theta + beam.bearing
        end_range  = min(beam.range, MAX_RANGE)
        end_x = x + end_range * math.cos(beam_angle)
        end_y = y + end_range * math.sin(beam_angle)

        gx0, gy0 = world_to_grid(x, y)
        gx1, gy1 = world_to_grid(end_x, end_y)
        if not inside_grid(gx0, gy0):
            return

        cells = bresenham_cells(gx0, gy0, gx1, gy1)
        if not cells:
            return

        # All cells except the last are free (ray passed through them)
        free_cells = cells[:-1] if beam.hit else cells
        for gx, gy in free_cells:
            if inside_grid(gx, gy):
                self.log_odds[gy, gx] = np.clip(
                    self.log_odds[gy, gx] + LOG_ODDS_FREE, LOG_ODDS_MIN,
   LOG_ODDS_MAX)

        # Terminal cell is occupied if the beam actually hit something
        if beam.hit and inside_grid(gx1, gy1):
            self.log_odds[gy1, gx1] = np.clip(
                self.log_odds[gy1, gx1] + LOG_ODDS_OCC, LOG_ODDS_MIN,
   LOG_ODDS_MAX)
```

```
# ====================================
# ICP scan-matching helpers
# ====================================

def icp_get_occupied_points(grid: OccupancyGridMap) -> np.ndarray:
    """Return world-frame (x, y) coordinates of occupied cells in the grid
    ."""
    occ_y, occ_x = np.nonzero(grid.log_odds > ICP_OCC_LOGODDS_THRESHOLD)
    if len(occ_x) == 0:
        return np.zeros((0, 2), dtype=float)
    if ICP_TARGET_DOWNSAMPLE > 1:
        occ_x = occ_x[::ICP_TARGET_DOWNSAMPLE]
        occ_y = occ_y[::ICP_TARGET_DOWNSAMPLE]
    xs = WORLD_XMIN + (occ_x.astype(float) + 0.5) * GRID_RES
    ys = WORLD_YMIN + (occ_y.astype(float) + 0.5) * GRID_RES
    return np.column_stack((xs, ys))


def icp_scan_to_robot_frame(beams: list) -> np.ndarray:
    """Convert hit beams into (x, y) endpoints in the robot body frame."""
    pts = []
    for beam in beams:
        if beam.hit and beam.range < 0.98 * MAX_RANGE:
            pts.append([beam.range * math.cos(beam.bearing),
                        beam.range * math.sin(beam.bearing)])
    return np.asarray(pts, dtype=float) if pts else np.zeros((0, 2), dtype
    =float)


def icp_transform_to_world(pose: np.ndarray, pts_local: np.ndarray) -> np.
    ndarray:
    """Rotate and translate robot-frame points into the world frame."""
    if len(pts_local) == 0:
        return np.zeros((0, 2), dtype=float)
    c, s = math.cos(pose[2]), math.sin(pose[2])
    R = np.array([[c, -s], [s, c]], dtype=float)
    return pts_local @ R.T + pose[:2]


def icp_nearest_correspondences(source: np.ndarray, target: np.ndarray,
                                max_dist: float = ICP_MAX_CORRESP_DIST):
    """
    Brute-force nearest-neighbour matching between source and target point
    sets.
    Returns matched (source, target, distances) rows within max_dist.
    """
    if len(source) == 0 or len(target) == 0:
        empty = np.zeros((0, 2), dtype=float)
        return empty, empty, np.zeros(0)
    diff = source[:, None, :] - target[None, :, :]
    d2   = np.sum(diff * diff, axis=2)
    nn   = np.argmin(d2, axis=1)
    best = d2[np.arange(len(source)), nn]
    keep = best <= max_dist ** 2
```

```
    return source[keep], target[nn[keep]], np.sqrt(best[keep])


def icp_solve_rigid_2d(source: np.ndarray, target: np.ndarray):
    """
    SVD Procrustes solution: find (R, t, dtheta) minimising ||R*src + t -
   tgt||.
    Returns the 2x2 rotation matrix, translation vector, and scalar
   rotation angle.
    """
    if len(source) < 2:
        return np.eye(2), np.zeros(2), 0.0
    cs, ct = np.mean(source, axis=0), np.mean(target, axis=0)
    H = (source - cs).T @ (target - ct)
    try:
        U, _, Vt = np.linalg.svd(H)
    except np.linalg.LinAlgError:
        return np.eye(2), np.zeros(2), 0.0
    R = Vt.T @ U.T
    if np.linalg.det(R) < 0.0:
        Vt[-1] *= -1.0
        R = Vt.T @ U.T
    t      = ct - R @ cs
    dtheta = math.atan2(float(R[1, 0]), float(R[0, 0]))
    return R, t, dtheta


def icp_scan_match_pose(grid: OccupancyGridMap,
                        pose_pred: np.ndarray,
                        beams: list) -> tuple:
    """
    Align the current scan against occupied cells in the grid using point-
   to-point ICP.

    Each iteration:
      1. Transform scan endpoints into the world frame using the current
   pose estimate.
      2. Find nearest-neighbour correspondences to occupied map cells.
      3. Solve the rigid 2D transform via SVD Procrustes.
      4. Clamp the step and update the pose estimate.

    Returns (matched_pose, score, n_matches).
    score = -RMSE (higher is better). Returns pose_pred unchanged on
   failure.
    """
    target     = icp_get_occupied_points(grid)
    scan_local = icp_scan_to_robot_frame(beams)

    if len(target) < ICP_MIN_MATCHES or len(scan_local) < ICP_MIN_MATCHES:
        return pose_pred.copy(), -float('inf'), 0

    current     = pose_pred.copy()
    final_dists = np.zeros(0)
    final_n     = 0
```

```
    for _ in range(ICP_MAX_ITER):
        source_world            = icp_transform_to_world(current,
    scan_local)
        src, tgt, dists         = icp_nearest_correspondences(source_world
    , target)
        final_dists, final_n    = dists, len(src)

        if final_n < ICP_MIN_MATCHES:
            break

        _, t_delta, dtheta = icp_solve_rigid_2d(src, tgt)

        # Clamp each iteration step for robustness
        dtheta = float(np.clip(wrap_angle(dtheta),
                               -ICP_MAX_ROTATION_STEP,
    ICP_MAX_ROTATION_STEP))
        t_norm = float(np.linalg.norm(t_delta))
        if t_norm > ICP_MAX_TRANSLATION_STEP:
            t_delta *= ICP_MAX_TRANSLATION_STEP / max(t_norm, 1e-12)

        current[:2] += t_delta
        current[2]   = wrap_angle(current[2] + dtheta)

        if t_norm < 0.004 and abs(dtheta) < math.radians(0.05):
            break

    if final_n == 0:
        return pose_pred.copy(), -float('inf'), 0

    rmse = float(np.sqrt(np.mean(final_dists ** 2))) if len(final_dists)
    else float('inf')
    return current, -rmse, int(final_n)
```

4.1.7 world_markers.py

```
#Provided directly from instructor Dr. Duffaut for this project (
    course_ode_plant_urdf_goal_world)



#!/usr/bin/env python3
import os
import yaml
import rclpy
from rclpy.node import Node

from visualization_msgs.msg import Marker
from geometry_msgs.msg import Point


def load_world(yaml_path: str):
    with open(yaml_path, 'r') as f:
        data = yaml.safe_load(f)
```

```
    w = data.get('world', {})
    walls = w.get('walls', [])
    boxes = w.get('boxes', [])
    return walls, boxes


def box_to_walls(cx, cy, ww, hh):
    x1 = cx - ww / 2.0
    x2 = cx + ww / 2.0
    y1 = cy - hh / 2.0
    y2 = cy + hh / 2.0
    return [
        [x1, y1, x2, y1],
        [x2, y1, x2, y2],
        [x2, y2, x1, y2],
        [x1, y2, x1, y1],
    ]


class WorldMarkers(Node):
    """
    Publish RViz Marker (LINE_LIST) for the world walls/boxes.
    Frame: odom (so it aligns with TF odom->base_footprint).
    """

    def __init__(self):
        super().__init__('world_markers')

        self.declare_parameter('world_yaml', '')
        self.declare_parameter('frame_id', 'odom')
        self.declare_parameter('topic', '/world_markers')
        self.declare_parameter('line_width', 0.03)

        world_yaml = self.get_parameter('world_yaml').value
        if not world_yaml or not os.path.exists(world_yaml):
            raise RuntimeError(f"world_yaml not found: {world_yaml}")

        self.frame_id = self.get_parameter('frame_id').value
        self.topic = self.get_parameter('topic').value
        self.line_width = float(self.get_parameter('line_width').value)

        walls, boxes = load_world(world_yaml)

        # Merge boxes into wall segments for visualization
        segs = list(walls)
        for b in boxes:
            segs += box_to_walls(float(b['cx']), float(b['cy']), float(b['
   w']), float(b['h']))

        self.marker = Marker()
        self.marker.header.frame_id = self.frame_id
        self.marker.ns = "world"
        self.marker.id = 0
        self.marker.type = Marker.LINE_LIST
```

```
        self.marker.action = Marker.ADD
        self.marker.scale.x = self.line_width

        # Color (neutral light gray)
        self.marker.color.r = 0.8
        self.marker.color.g = 0.8
        self.marker.color.b = 0.8
        self.marker.color.a = 1.0

        # Build points
        pts = []
        for s in segs:
            x1, y1, x2, y2 = [float(v) for v in s]
            p1 = Point(x=x1, y=y1, z=0.0)
            p2 = Point(x=x2, y=y2, z=0.0)
            pts += [p1, p2]
        self.marker.points = pts

        self.pub = self.create_publisher(Marker, self.topic, 1)
        self.timer = self.create_timer(0.5, self.tick)

        self.get_logger().info(f"Publishing world markers on {self.topic}
   in frame {self.frame_id}")

    def tick(self):
        self.marker.header.stamp = self.get_clock().now().to_msg()
        self.pub.publish(self.marker)


def main():
    rclpy.init()
    n = WorldMarkers()
    rclpy.spin(n)
    n.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

4.1.8 launch.py

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    pkg_name = 'ekf_slam'

    # Define paths to URDF, RViz config, and World YAML files
    urdf_path = os.path.join(get_package_share_directory(pkg_name), 'urdf'
   , 'tri_wheel.urdf')
    rviz_cfg = os.path.join(get_package_share_directory(pkg_name), 'rviz',
```

```
    'slam_view.rviz')
    world_yaml = os.path.join(get_package_share_directory(pkg_name), '
config', 'simple_world.yaml')

    with open(urdf_path, 'r') as f:
        robot_desc = f.read()

    return LaunchDescription([
        # 1. ODE Plant - From HW5 Solutions
        Node(
            package=pkg_name,
            executable='omni_ode_plant',
            name='omni_ode_plant',
            parameters=[{'dt': 0.05, 'pose_topic': '/pose', 'cmd_topic': '
/cmd_vel'}]
        ),

        # 2. TF Broadcaster - From HW5 Solutions
        Node(
            package=pkg_name,
            executable='ode_tf_broadcaster',
            name='ode_tf_broadcaster',
            parameters=[{'pose_topic': '/pose', 'odom_frame': 'odom', '
base_frame': 'base_footprint'}]
        ),

        # 3. Robot State Publisher - Standard Node
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            parameters=[{'robot_description': robot_desc}]
        ),

        # 4. Joint State Publisher - Standard Node
        Node(
            package='joint_state_publisher',
            executable='joint_state_publisher',
            name='joint_state_publisher'
        ),

        # 5. Fake Lidar Scanner - From provided
course_ode_plant_urdf_goal_world
        Node(
            package=pkg_name,
            executable='fake_lidar_scan_world',
            name='fake_lidar_scan_world',
            parameters=[{
                'scan_topic': '/scan',
                'lidar_frame': 'lidar_link',
                'odom_frame': 'odom',
                'world_yaml': world_yaml,
                'num_rays': 360,
                'range_max': 5.0
```

```
            }]
        ),

        # 6. World Markers - From provided
    course_ode_plant_urdf_goal_world
        Node(
            package=pkg_name,
            executable='world_markers',
            name='world_markers',
            parameters=[{'world_yaml': world_yaml, 'frame_id': 'odom', '
    topic': '/world_markers'}]
        ),

        # 7. EKF SLAM Node - Created for this Project
        Node(
            package=pkg_name,
            executable='ekf_node',
            name='ekf_node',
            output='screen'
        ),

        # 8. RViz2 - Standard Node
        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
            arguments=['-d', rviz_cfg]
        ),

       # 9. Omni-directional Go-To-Goal Node - From HW5 Solutions
        Node(
            package=pkg_name,
            executable='omni_go_to_goal',
            name='omni_go_to_goal',
            output='screen',
            parameters=[{
                'k_xy': 1.2,
                'k_theta': 3.0,
                'vxy_max': 0.7,
                'w_max': 2.0,
                'goal_tol_xy': 0.08,
                'pose_topic': '/pose',
                'cmd_topic': '/cmd_vel',
                'goal_pose_topic': '/goal_pose'
            }]
        ),

    ])
```

4.1.9 slam_view.rviz

```
# Adapted from provided world_scan_map.rviz file
```

```
Panels:
  - Class: rviz_common/Displays
    Help Height: 78
    Name: Displays
    Property Tree Widget:
      Expanded:
        - /Global Options1
        - /RobotModel1
        - /LaserScan1
        - /Map1
      Splitter Ratio: 0.5
    Tree Height: 549
  - Class: rviz_common/Views
    Expanded:
      - /Current View1
    Name: Views
    Splitter Ratio: 0.5
Visualization Manager:
  Class: ""
  Displays:

    # global grid
    - Class: rviz_default_plugins/Grid
      Enabled: true
      Name: Grid
      Value: true
      Alpha: 0.5
      Cell Size: 1
      Color:
        R: 0.5
        G: 0.5
        B: 0.5
        A: 1.0
      Line Style:
        Line Width: 0.03
        Value: Lines
      Normal Cell Count: 0
      Offset:
        X: 0
        Y: 0
        Z: 0
      Plane: XY
      Plane Cell Count: 20
      Reference Frame: <Fixed Frame>

    # TF tree
    - Class: rviz_default_plugins/TF
      Enabled: true
      Name: TF
      Value: true
      Frame Timeout: 15
      Frames:
        All Enabled: true
      Marker Scale: 0.3
```

```
    Show Arrows: true
    Show Axes: true
    Show Names: true
    Update Interval: 0

  # robot model
  - Class: rviz_default_plugins/RobotModel
    Enabled: true
    Name: RobotModel
    Value: true
    Alpha: 1
    Description Topic: /robot_description
    Description Source: Topic
    Visual Enabled: true
    Collision Enabled: false
    Robot Description: ""
    TF Prefix: ""
    Update Interval: 0

  # 4world walls
  - Class: rviz_default_plugins/Marker
    Enabled: true
    Name: World
    Value: true
    Namespaces:
      world: true
    Topic:
      Depth: 5
      Durability Policy: Volatile
      Filter size: 10
      History Policy: Keep Last
      Reliability Policy: Reliable
      Value: /world_markers

  # lidar scan
  - Class: rviz_default_plugins/LaserScan
    Enabled: true
    Name: LaserScan
    Value: true
    Topic:
      Depth: 5
      Durability Policy: Volatile
      Filter size: 10
      History Policy: Keep Last
      Reliability Policy: Best Effort
      Value: /scan
    Autocompute Intensity Bounds: true
    Autocompute Value Bounds:
      Max Value: 10
      Min Value: 0
      Value: true
    Axis: Z
    Channel Name: intensity
    Color:
```

```
      R: 0.0
      G: 1.0
      B: 0.0
      A: 1.0
    Color Transformer: FlatColor
    Decay Time: 0
    Position Transformer: XYZ
    Queue Size: 10
    Size (Pixels): 3
    Size (m): 0.05
    Style: Points
    Use Fixed Frame: false

  # occupancy grid map
  - Class: rviz_default_plugins/Map
    Enabled: true
    Name: Map
    Value: true
    Topic:
      Depth: 1
      Durability Policy: Volatile
      Filter size: 10
      History Policy: Keep Last
      Reliability Policy: Reliable
      Value: /map
    Alpha: 0.7
    Color Scheme: map
    Draw Behind: false
    Resolution: 0.1

Enabled: true
Global Options:
  Background Color:
    R: 0.05
    G: 0.05
    B: 0.05
    A: 1.0
  Fixed Frame: odom
  Frame Rate: 30

Name: root
Tools:
  - Class: rviz_default_plugins/Interact
    Hide Inactive: false
  - Class: rviz_default_plugins/MoveCamera
  - Class: rviz_default_plugins/Select
  - Class: rviz_default_plugins/FocusCamera
  - Class: rviz_default_plugins/Measure
    Line color: 128; 128; 0
  - Class: rviz_default_plugins/SetInitialPose
    Covariance x: 0.25
    Covariance y: 0.25
    Covariance yaw: 0.068538919
    Topic:
```

```
      Depth: 5
      Durability Policy: Volatile
      Filter size: 10
      History Policy: Keep Last
      Reliability Policy: Reliable
      Value: /initialpose
    - Class: rviz_default_plugins/SetGoal
      Topic:
      Depth: 5
      Durability Policy: Volatile
      Filter size: 10
      History Policy: Keep Last
      Reliability Policy: Reliable
      Value: /goal_pose

  Value: true

  Views:
    Current:
      Class: rviz_default_plugins/TopDownOrtho
      Enable Stereo Rendering:
        Stereo Eye Separation: 0.06
        Stereo Focal Distance: 1
        Swap Stereo Eyes: false
        Value: false
      Invert Z Axis: false
      Name: Current View
      Near Clip Distance: 0.01
      Scale: 50
      Target Frame: <Fixed Frame>
      Value: TopDownOrtho (rviz_default_plugins)
      X: 0
      Y: 0
    Saved: ~
```

4.1.10 tri_wheel.urdf

```
<?xml version="1.0"?>
<robot name="tri_wheel_omni_enhanced">

  <material name="roller_gray">
    <color rgba="0.45 0.45 0.45 1.0"/>
  </material>

  <material name="chassis_gray">
    <color rgba="0.65 0.65 0.65 1.0"/>
  </material>

  <material name="mount_dark">
    <color rgba="0.25 0.25 0.25 1.0"/>
  </material>

  <material name="wheel_black">
    <color rgba="0.08 0.08 0.08 1.0"/>
```

```
</material>

<material name="x_red">
  <color rgba="0.85 0.10 0.10 1.0"/>
</material>

<link name="base_footprint"/>

<link name="base_link">
  <visual>
    <origin xyz="0 0 0.095" rpy="0 0 0"/>
    <geometry>
      <cylinder radius="0.14" length="0.10"/>
    </geometry>
    <material name="chassis_gray"/>
  </visual>
  <collision>
    <origin xyz="0 0 0.095" rpy="0 0 0"/>
    <geometry>
      <cylinder radius="0.14" length="0.10"/>
    </geometry>
  </collision>
  <inertial>
    <origin xyz="0 0 0.095" rpy="0 0 0"/>
    <mass value="3.0"/>
    <inertia ixx="0.02" ixy="0.0" ixz="0.0" iyy="0.02" iyz="0.0" izz
 ="0.015"/>
  </inertial>
</link>

<joint name="base_footprint_to_base_link" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
</joint>

<link name="x_axis_indicator_link">
  <visual>
    <origin xyz="0.10 0.0 0.155" rpy="0 1.57079632679 0"/>
    <geometry>
      <cylinder radius="0.008" length="0.20"/>
    </geometry>
    <material name="x_red"/>
  </visual>
</link>

<joint name="x_axis_indicator_joint" type="fixed">
  <parent link="base_link"/>
  <child link="x_axis_indicator_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
</joint>

<link name="wheel_1_mount_link">
  <visual>
```

```
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <box size="0.06 0.02 0.02"/>
    </geometry>
    <material name="mount_dark"/>
  </visual>
</link>

<joint name="wheel_1_mount_joint" type="fixed">
  <parent link="base_link"/>
  <child link="wheel_1_mount_link"/>
  <origin xyz="0.175 0.0 0.090" rpy="0 0 0"/>
</joint>

<link name="wheel_1_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 1.57079632679 0"/>
    <geometry>
      <cylinder radius="0.075" length="0.025"/>
    </geometry>
    <material name="wheel_black"/>
  </visual>
  <visual><origin xyz="0 0.075 0.000" rpy="0 0 0"/><geometry><cylinder
 radius="0.010" length="0.030"/></geometry><material name="roller_gray
 "/></visual>
  <visual><origin xyz="0 0.053033 0.053033" rpy="0.78539816339 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.000 0.075" rpy="1.57079632679 0 0"/><geometry
 ><cylinder radius="0.010" length="0.030"/></geometry><material name="
 roller_gray"/></visual>
  <visual><origin xyz="0 -0.053033 0.053033" rpy="2.35619449019 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 -0.075 0.000" rpy="3.14159265359 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 -0.053033 -0.053033" rpy="3.92699081699 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.000 -0.075" rpy="4.71238898038 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.053033 -0.053033" rpy="5.49778714378 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
</link>

<joint name="wheel_1_joint" type="continuous">
  <parent link="wheel_1_mount_link"/>
  <child link="wheel_1_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <axis xyz="1 0 0"/>
</joint>
```

```
<link name="wheel_2_mount_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry><box size="0.06 0.02 0.02"/></geometry>
    <material name="mount_dark"/>
  </visual>
</link>

<joint name="wheel_2_mount_joint" type="fixed">
  <parent link="base_link"/>
  <child link="wheel_2_mount_link"/>
  <origin xyz="-0.0875 0.151553 0.090" rpy="0 0 2.09439510239"/>
</joint>

<link name="wheel_2_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 1.57079632679 0"/>
    <geometry><cylinder radius="0.075" length="0.025"/></geometry>
    <material name="wheel_black"/>
  </visual>
  <visual><origin xyz="0 0.075 0.000" rpy="0 0 0"/><geometry><cylinder
 radius="0.010" length="0.030"/></geometry><material name="roller_gray
 "/></visual>
  <visual><origin xyz="0 0.053033 0.053033" rpy="0.78539816339 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.000 0.075" rpy="1.57079632679 0 0"/><geometry
 ><cylinder radius="0.010" length="0.030"/></geometry><material name="
 roller_gray"/></visual>
  <visual><origin xyz="0 -0.053033 0.053033" rpy="2.35619449019 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 -0.075 0.000" rpy="3.14159265359 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 -0.053033 -0.053033" rpy="3.92699081699 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.000 -0.075" rpy="4.71238898038 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.053033 -0.053033" rpy="5.49778714378 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
</link>

<joint name="wheel_2_joint" type="continuous">
  <parent link="wheel_2_mount_link"/>
  <child link="wheel_2_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <axis xyz="1 0 0"/>
</joint>
```

```
<link name="wheel_3_mount_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry><box size="0.06 0.02 0.02"/></geometry>
    <material name="mount_dark"/>
  </visual>
</link>

<joint name="wheel_3_mount_joint" type="fixed">
  <parent link="base_link"/>
  <child link="wheel_3_mount_link"/>
  <origin xyz="-0.0875 -0.151553 0.090" rpy="0 0 4.18879020479"/>
</joint>

<link name="wheel_3_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 1.57079632679 0"/>
    <geometry><cylinder radius="0.075" length="0.025"/></geometry>
    <material name="wheel_black"/>
  </visual>
  <visual><origin xyz="0 0.075 0.000" rpy="0 0 0"/><geometry><cylinder
 radius="0.010" length="0.030"/></geometry><material name="roller_gray
 "/></visual>
  <visual><origin xyz="0 0.053033 0.053033" rpy="0.78539816339 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.000 0.075" rpy="1.57079632679 0 0"/><geometry
 ><cylinder radius="0.010" length="0.030"/></geometry><material name="
 roller_gray"/></visual>
  <visual><origin xyz="0 -0.053033 0.053033" rpy="2.35619449019 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 -0.075 0.000" rpy="3.14159265359 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 -0.053033 -0.053033" rpy="3.92699081699 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.000 -0.075" rpy="4.71238898038 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
  <visual><origin xyz="0 0.053033 -0.053033" rpy="5.49778714378 0 0"/><
 geometry><cylinder radius="0.010" length="0.030"/></geometry><material
 name="roller_gray"/></visual>
</link>

<joint name="wheel_3_joint" type="continuous">
  <parent link="wheel_3_mount_link"/>
  <child link="wheel_3_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <axis xyz="1 0 0"/>
</joint>

<link name="lidar_link">
```

```
    <visual>
      <geometry><cylinder radius="0.03" length="0.04"/></geometry>
      <material name="black"><color rgba="0.1 0.1 0.1 1"/></material>
    </visual>
  </link>

  <joint name="lidar_joint" type="fixed">
    <parent link="base_link"/>
    <child link="lidar_link"/>
    <!-- Mount lidar at center, slightly above the base -->
    <origin xyz="0 0 0.05" rpy="0 0 0"/>
  </joint>

</robot>
```

4.1.11 package.xml

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
   schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>ekf_slam</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="22johpoiri@gmail.com">jlpoirie</maintainer>
  <license>TODO: License declaration</license>

  <depend>rclpy</depend>
  <depend>std_msgs</depend>
  <depend>geometry_msgs</depend>
  <depend>sensor_msgs</depend>
  <depend>nav_msgs</depend>
  <depend>visualization_msgs</depend>
  <depend>tf2_ros</depend>
  <depend>turtlesim</depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

4.1.12 setup.py

```
import os
from glob import glob
from setuptools import setup

package_name = 'ekf_slam'
```

```
setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        # Install launch files
        (os.path.join('share', package_name, 'launch'), glob('launch/*.py'
   )),
        # Install URDF files
        (os.path.join('share', package_name, 'urdf'), glob('urdf/*.urdf'))
   ,
        # Install config/world files
        (os.path.join('share', package_name, 'config'), glob('config/*.
   yaml')),
        # Install RViz config files
        (os.path.join('share', package_name, 'rviz'), glob('rviz/*.rviz'))
   ,
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='John Poirier',
    maintainer_email='22johpoiri@gmail.com',
    description='ROS2 EKF SLAM for tri-wheel robot',
    license='Apache License 2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'ekf_node = ekf_slam.ekf_node:main',
            'fake_lidar_scan_world = ekf_slam.fake_lidar_scan_world:main',
            'ode_tf_broadcaster = ekf_slam.ode_tf_broadcaster:main',
            'omni_ode_plant = ekf_slam.omni_ode_plant:main',
            'world_markers = ekf_slam.world_markers:main',
            'omni_go_to_goal = ekf_slam.omni_go_to_goal:main',
        ],
    },
)
```